Unit 3. Pointers & Arrays

3.0 Array Syntax

Sam is working on a function to compute a letter grade from a number grade. While this can be easily done using IF/ELSE statements, he feels there must be an easier way. There is a pattern in the numbers which he should be able to leverage to make for a more elegant and efficient solution. While mulling over this problem, Sue introduces him to arrays…

Objectives

By the end of this class, you will be able to:

- Declare an array to solve a problem.
- Write a loop to traverse an array.
- Predict the output of a code fragment containing an array.
- Pass an array to a function.

Prerequisites

Before reading this section, please make sure you are able to:

- Demonstrate the correct syntax for a WHILE, DO-WHILE, and FOR loop (Chapter 2.3).
- Create a loop to solve a simple problem (Chapter 2.3).

Overview

In the simplest form, an array is a "bucket of variables." Rather than having many variables to represent the values in a collection, we can have a single variable representing the bucket. There are many instances when working with buckets is more convenient than working with individual data items. One example is text:

```
char text[256];
```

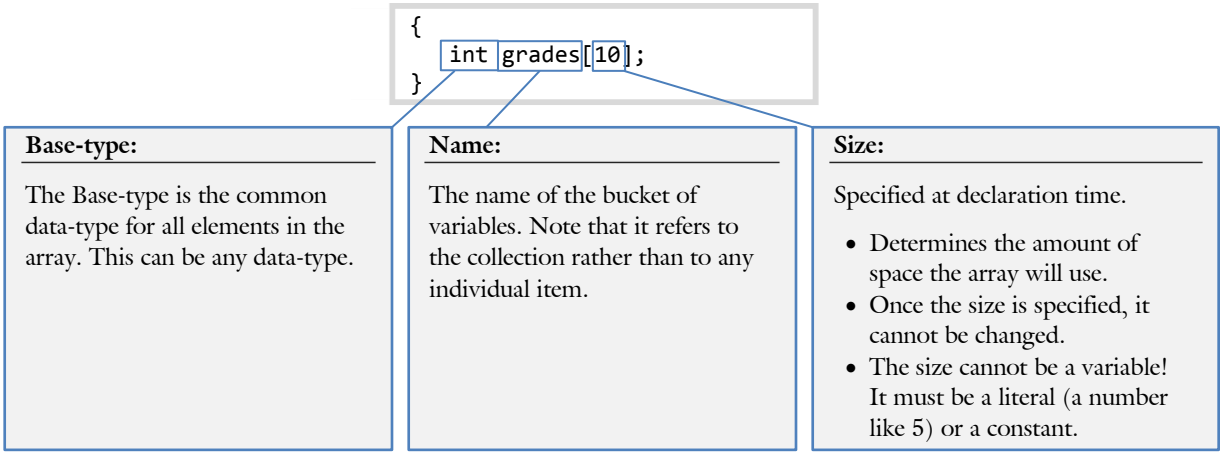
In this example, the important unit is the collection of characters rather than any single character. It would be extremely inconvenient to have to manage 256 individual variables to store the data of a single string. There are three main components of the syntax of an array: the syntax for declaring an array, the syntax for referencing an individual item in an array, and the syntax for passing an array as a parameter:

Declaring an array	Referencing an array	Passing as a parameter

Syntax:
`<Type> <variable>[size]`
Example:
`int grades[200];`
A few details:
• Any data-type can be used.
• The size must be a natural number {1, 2, etc.}, not a variable.
	Syntax:
	`<variable>[<index>]`
	Example:
	`cout << grades[i];`
	A few details:
	• The index starts with 0 and must be within the valid range.
		Syntax:
		`(<Type> <variable>[])`
		Example:
		`void func(int grades[])`
		A few details:
		• You must specify the base-type.
		• No size is passed in the square brackets `[]`.

Declaring an Array

A normal variable declaration asks the compiler to reserve the necessary amount of memory and allows the user to reference the memory by the variable name. Arrays are slightly different. The amount of memory reserved is computed by the size of each member in the list multiplied by the number of items in the list.



The first part is the base-type. This is the type of data common to all items in the list. In other words, we can't have an array where some items are integers and others are characters. We can use any data-type as the base-type. This includes built-in data-types (`int`, `float`, `bool`, etc.) as well as custom data-types we will create in future semesters.

The second part is the name. Since the array name refers to the collection of elements (as opposed to any individual element), this name is commonly plural. Another common naming convention is to have the "`list`" prefix (`int listGrade[10];`).

The final part is the size. It is important to note that the compiler needs to know the size of the array at compilation time. In other words, we cannot make this a variable that the user provides the value for. It is legal to have a literal (example: `10`), a constant earlier defined (example: `const int SIZE = 10;`) or a `#define` resolving to a constant or a literal (example: `#define SIZE 10`). One final note: once the size has been specified, it cannot be changed. We will learn how to specify the size at run-time using a variable later in the semester (Chapter 4.1)

Sam's Corner

While it is illegal to have a variable in the square brackets of an array declaration, our compiler lets it slide. It is a very bad idea to rely on a given compiler's non-adherence to the language standard: it will make it difficult to port (or move) the code to another compiler.

That being said, the new C++ standard (called C++11) makes an allowance on this front. Please read about generalized constant expressions: C++11 Generalized constant expressions.

Initializing

When declaring a simple variable, it is possible to initialize the variable at the same time:

```
{
   int variable1;             // declared but uninitialized
   variable1 = 10;            // now it is initialized

   int variable2 = 10;        // declared and initialized in one step
}
```

It is also possible to declare and initialize an array variable in one step:

```
{
   char grades[4] =
   {
      'A', 'C', 'B', 'A'
   };
}
```

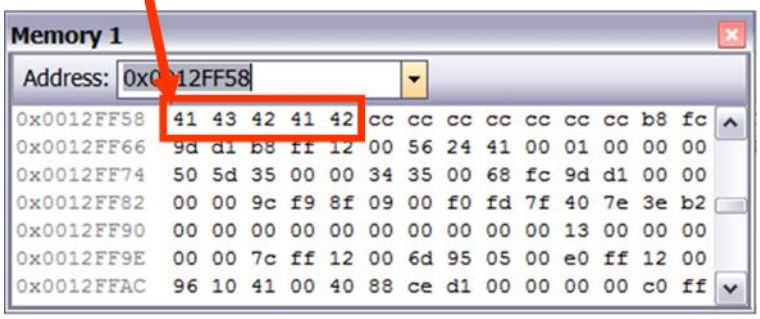
Observe how the list of items to initialize are delimited with curly braces (`{}`). Since the Elements of Style demands that each curly brace be on its own line, we align them with the base-type. Finally, the individual items in the array are presented in a comma-separated list.

Declaration	In memory	Description
`int array[6];`	? ? ? ? ? ?	Though six slots were set aside, they remain uninitialized. All slots are filled with unknown values.
`int array[6] = { 3, 6, 2, 9, 1, 8 };`	3 6 2 9 1 8	The initialized size is the same as the declared size so every slot has a known value.
`int array[6] = { 3, 6 };`	3 6 0 0 0 0	The first 2 slots are initialized, the balance are filled with 0. This is a partially filled array.
`int array[] = { 3, 6, 2, 9, 1, 8 };`	3 6 2 9 1 8	Declared to exactly the size necessary to fit the list of numbers. The compiler will count the number of slots
`int array[6] = {};`	0 0 0 0 0 0	This is the easiest way to initialize an array with zeros in all the slots

Sam's Corner

Arrays are stored in memory in the most efficient way imaginable: just a continuous block of adjacent memory locations. The array variable itself refers to (or "points to") the first item in that block of memory. Consider the following memory-dump of a simple declaration of an array of characters:

```
char grades[5] = {'A', 'C', 'B', 'A', 'B'};
```



In this example, the memory starts at location `0x0012FF58`.

Declaring an array of strings

Since arrays of characters are called strings, how do we make arrays of strings? In essence, we will need an array of arrays. We call these multi-dimensional arrays and they will be the topic of Chapter 4.0.

```
{
   char listNames[10][256];      // ten strings
}
```

Observe the two sets of square brackets. The first set (`[10]`) refers to the number of strings in the array of strings. The second set (`[256]`) refers to the size of each individual string in the list. As a result, we will have ten strings, each 256 bytes in length. This means the total size of `listNames` is `sizeof(char) * 10 * 256`.

We can also initialize an array of strings at declaration time:

```
{
   char listNames[][256] =       // the number of strings is not necessary,
   {                             //   the compiler can count
      "Thomas",
      "Edwin",                   // use either "quotes" or {'E', 'd', 'w', … },
      "Ricks"
   };
}
```

Referencing an Array

In mathematics, we can define a sequence of numbers much like we define arrays of numbers in C++. We refer to individual members of a sequence in mathematics with the subscript notation: X_2 is the second element of the sequence *X*. We use the square bracket notation in C++:

```
cout << list[3] << endl;          // Access the fourth item in the list
```

One important difference between arrays and mathematical sequences is that the indexing of arrays starts at zero. In other words, the first item is `list[0]` and the second is `list[1]`. The reason for this stems from how arrays are declared. Recall that the array variable refers to (or points to) the first item in the list. The array index is actually the offset from that first item. Thus, when one references `list[2]`, one is actually saying "move 2 spots from the first item."

Loops

Since indexing for arrays starts at zero, the valid indices for an array of 10 items is 0 … 9. This brings us to our standard FOR loop for arrays:

```
for (int i = 0; i < num; i++)
   cout << list[i];
```

From this loop we notice several things. First, the array index variable is commonly the letter `i` or some version of it (such as `iList`). This is one of the few times we can get away with a one letter variable name.

The second point is that we typically start the list at zero. If we start at one, we will skip the first item in the list.

The Boolean expression is (`i < num`). Observe how we could also say (`i <= num - 1`). This, however, is needlessly complex. We can read the Boolean expression as "as long as the counter is less than the number of items in the list."

Sue's Tips

It is very bad to index off the end of an array. If, for example, we have an array of 10 numbers, what happens when we attempt to access the 20th item?

```
{
   int list[10];
   int i = 20;
   list[i] = -1;              // error!  Off the end of the list
}
```

The compiler will not prevent such program logic mistakes; it is up to the programmer to catch these errors. In the above example, we will assign the value `-1` to some random location of memory. This will probably cause the program to malfunction in an unpredictable way. The best way to prevent this class of problems is to use asserts to verify our referencing:

```
{
   int list[10];
   int i = 20;

   assert(i >= 0 && i < 10);  // much easier bug to fix
   list[i] = -1;
}
```

Always use an assert to verify that the index is in the range of legal values!

Referencing an array of strings

Arrays of strings are also referenced with the square brackets. However, the programmer needs to specify whether an individual character from a string is to be referenced, whether an entire string is to be referenced, or whether we are working with the collection of strings. Consider the following example:

```
{
   char listNames[][256] =
   {
      "Thomas",
      "Edwin",
      "Ricks"
   };

   cout << listNames[0][0] << endl;      // the letter 'T'
   cout << listNames[0]    << endl;      // the string "Thomas"
   cout << listNames       << endl;      // ERROR: COUT can't accept an array of
}                                        //        strings. Write a loop!
```

Example 3.0 – Array Copy

Demo

It is possible to copy the data from one integer variable into another with a simple assignment operator. This is not true with an array. To perform this task, a loop is required. This example will demonstrate how to copy an array of integers.

Problem

Write a program to prompt the user for a list of ten numbers. After the user has entered the values, copy the values into another array and display the list.

Solution

It is not possible to perform the array copy with a simple assignment statement:

```
arrayDestination = arraySource;   // this will not work
```

Instead, it is necessary to write a loop and copy the items one at a time.

```
{
   const int SIZE = 10;       // we can use SIZE to declare because it is a CONST
   int listDestination[SIZE]; // copy data to here. It starts uninitialized
   int listSource[SIZE] =     // copy data from here
   {
      6, 8, 2, 6, 1, 7, 2, 9, 0
   };

   // a FOR loop is required to copy the data from one array to another.
   for (int i = 0; i < SIZE; i++)
      listDestination[i] = listSource[i];
}
```

Challenge

As a challenge, modify the program to copy an array of floating point numbers rather than an array of integers. Make sure you format the output to one or two decimals of accuracy.

As an additional challenge, try to display the list backwards.

See Also

The complete solution is available at 3-0-arrayCopy.cpp or:

```
/home/cs124/examples/3-0-arrayCopy.cpp
```

Arrays as Parameters

Passing arrays as parameters is quite different than passing other data types. The reason for this is a bit subtle. When passing an integer, a copy of the value is sent to the callee. When passing an array, however, the data itself does not move. Instead, only the address of the data is sent. This means, in effect, that passing arrays is always pass-by-reference.

Passing strings

As mentioned previously, strings are just arrays. Thus, passing a string as a parameter is the same as passing an array as a parameter. For any parameter-passing scenario, there are two parts: the callee (the function being called) and the caller (the function initiating the function call).

The parameter declaration in the callee looks much like the declaration of an array. There is one exception however: there is no number inside the square brackets. The reason for this may seem a bit counter-intuitive at first: the callee does not know the size of the array. Consider the following example:

```
/*****************************************
 * DISPLAY NAME
 * Display a user's name on the screen
 *****************************************/
void displayName(char lastName[], bool isMale) // no number inside the brackets!
{
   if (isMale)
      cout << "Brother ";
   else
      cout << "Sister ";
   cout << lastName;                          // treated like any other string
   return;
}
```

In this example, the first parameter (`lastName`) is a string. For the rest of the function, we can treat `lastName` like any local variable in the function.

Notice that we use the parameter mechanism to pass data back to the caller rather than using the return mechanism. We do this because array parameters behave like pass-by-reference variables.

```
/*****************************************
 * GET NAME
 * Prompt the user for this last name
 *****************************************/
void getName(char lastName[])              // Even though this is pass-by-reference
{                                          //     there is no & in the parameter
   cout << "What is your last name? ";
   cin  >> lastName;
   return;                                 // Do not use the return mechanism
}                                          //     for passing arrays
```

Observe how both the input parameter (demonstrated in the function `displayName()`) and the output parameter (demonstrated in the function `getName()`) pass the same way.

Calling a function accepting an array as a parameter is accomplished by passing the entire array name. Observe how we do not use square brackets when passing the string.

```
/*****************************************
 * MAIN
 * Driver function for displayName and getName
 *****************************************/
int main()
{
   char name[256];                              // this buffer is 256 characters
   getName(name);                               // no []s used when passing arrays

   displayName(name,    true  /*isMale*/);     // again, no []s
   displayName("Smith", false /*isMale*/);     // we can also pass a string literal.
                                                // this buffer is not 256 chars!
   return 0;
}
```

The complete program for this example is available on 3-0-passingString.cpp:

```
/home/cs124/examples/3-0-passingString.cpp
```

When calling a function with an array, do not use the square brackets (`[]`s). If you do, you will be sending only one element of the array (a `char` in this case). Observe how we can pass either a string variable (`name`) or a string literal (`"Smith"`). In the former case, the buffer is 256 characters. In the latter case, the buffer is much smaller. Therefore, the caller specifies the buffer size, not the callee. This is why the callee omits a number inside the square brackets (`[]`s) in the parameter declaration.

Sam's Corner

Recall that we should only be passing parameters by-reference when we want the callee to change the value. This gets a bit confusing because passing arrays as parameters is much like pass-by-reference. How can we avoid this unnecessarily tight coupling? The answer is to use the `const` modifier.

The `const` modifier allows the programmer to say "This variable will never change." When used in a parameter, it is a guarantee that the function will not alter the data in the variable. It would therefore be more correct to declare `displayName()` as follows:

```
void displayName(const char lastName[], bool isMale);
```

If the programmer made a mistake and actually tried to change the value in the function, the following compiler error message would be displayed:

```
example.cpp: In function "void displayName(const char*, bool)":
example.cpp:15: error: assignment of read-only location "* lastName"
```

Passing arrays

Passing arrays as parameters is much like passing strings with one major exception. While strings are frequently (but not always!) 256 characters in length, the size of array buffers are difficult to predict. For this reason, we almost always pass the size of an array along with the array itself:

```
/***************************************
 * FILL LIST
 * Fill a list with the user input
 ***************************************/
void fillList(float listGPAs[], int numGPAs)
{
   cout << "Please enter " << numGPAs << " GPAs\n";
   for (int iGPAs = 0; iGPAs < numGPAs; iGPAs++)
   {
      cout << "\t#" << iGPAs + 1 << " : ";
      cin  >> listGPAs[iGPAs];
   }
}
```

In this case, the function would not know the number of items in the list if the caller did not pass that value as a parameter.

Sue's Tips

There are commonly three variables in the typical array loop: the array to be looped through, the number of items in the array, and the counter itself. Each of these is related yet each fulfill a different role. It is a good idea to choose variable names to emphasize the differences and similarities:

- `listGPA`: The "`list`" prefix indicates it is an array, the "`GPA`" suffix indicates it pertains to GPAs.
- `numGPA`: The "`num`" prefix indicates it is the number of items, the "`GPA`" suffix again indicates what list it pertains to.
- `iGPA`: The "`i`" prefix indicates it is an incrementer (or iterator).

Using consistent and predictable names makes it easier to spot bugs.

Observe how the caller can then specify the size of the array and, by doing so, control the number of iterations through the loop:

```
/***************************************
 * MAIN
 * Driver program for fillList
 ***************************************/
int main()
{
   float listSmall[5];
   float listBig[500];

   fillList(listSmall, 5);   // make sure the passed size equals the true size
   fillList(listBig, 500);   // this time, many more iterations will be performed

   return 0;
}
```

A common mistake is to pass the wrong size of the list as a parameter. In the above example, it would be a mistake to pass the number `10` for the size of `listSmall` because it is only 5 slots in size.

Example 3.0 – Passing an Array

Demo

This example will demonstrate how to pass arrays of numbers as parameters. Included will be how to pass an array as an input parameter (to be filled) and how to pass an array as an output parameter (to be displayed).

Problem

Write a program to prompt the user for 4 prices in Euros, and display the dollar amount.

```
Please enter 4 prices in Euros
        Price # 1: 1.00
        Price # 2: 3.75
        Price # 3: .17
        Price # 4: 104.54
The prices in US dollars are:
        $1.38
        $5.17
        $0.23
        $144.04
```

Solution

The function to prompt the user for the prices is an output parameter. Notice how we do not use the ambersand (`&`) when specifying an output parameter if the parameter is an array.

```
void getPrices(float prices[], int num)
{
   cout << "Please enter " << num << " prices in Euros\n";

   for (int i = 0; i < num; i++)
   {
      cout << "\tPrice # " << i + 1 << ": ";
      cin  >> prices[i];
   }
}
```

When the parameter is input-only, it is a good idea to include the `const` modifier to the array parameter declaration to indicate that the function will not modify any of the data.

```
void display(const float prices[], int num)
{
   // configure the output for money
   cout.setf(ios::fixed);
   cout.setf(ios::showpoint);
   cout.precision(2);

   // display the prices
   cout << "The prices in US dollars are:\n";
   for (int i = 0; i < num; i++)
      cout << "\t$" << prices[i] << endl;
}
```

Challenge

As a challenge, modify the `display()` function so both the Euro and the Dollar amounts are displayed. This will require you to make a copy of the `price` array so both arrays can be passed to the display function.

See Also

The complete solution is available at 3-0-passingArray.cpp or:

```
/home/cs124/examples/3-0-passingArray.cpp
```

Sue's Tips

Always pass the size of the array as a parameter. The least problematic way to do this is to let the compiler compute the number of elements. Consider the following code:

```
float listSmall[5];
cout << sizeof(listSmall) << endl;      // sizeof(float) * 5 == 20
cout << sizeof(listSmall[0]) << endl;   // sizeof(float) == 4
cout << sizeof(listSmall) / sizeof(listSmall[0]) << endl;
                                        // 20 / 4 == 5 NumElements!
```

Thus, it is very common to pass the size of a list using the following convention:

```
fillList(listSmall, sizeof(listSmall) / sizeof(listSmall[0]) );
```

This expression is worth memorizing.

Passing an array of strings

Passing arrays of strings is a bit more complex than passing single strings. While the reason for the differences won't become apparent until we learn about multi-dimensional arrays later in the semester (Chapter 4.0), the syntax is as follows:

```
/*****************************************
 * DISPLAY NAMES
 * Display all the names in the passed list
 *****************************************/
void displayNames(char names[][256], int num)  // second [] has the size in it
{
   for (int i = 0; i < num; i++)               // same as with other arrays
      cout << names[i] << endl;                // access each individual string
}
```

Observe that, like with simple strings, the first set of square brackets near the `names` variable is empty. The second set, however, needs to include the size of each individual string. When we call this function, one does not include the square brackets. This ensures the complete list of names is passed, not an individual name.

```
/*****************************************
 * MAIN
 * Simple driver program for displayNames
 *****************************************/
int main()
{
   char fullName[3][256] =
   {
      "Thomas",
      "Edwin",
      "Ricks"
   };
   displayNames(fullName, 3);
   return 0;
}
```

Example 3.0 – Array of Strings

Demo

This example will demonstrate how to pass an array of strings as a parameter.

Problem

In this final example, we will prompt the user for his five favorite scripture heroes and display the results in a comma-separated list:

```
Who are your top five scripture heroes?
        #1: Joseph
        #2: Paul
        #3: Esther
        #4: Nephi
        #5: Mary
Your five heroes are: Joseph, Paul, Esther, Nephi, Mary
```

Solution

The function to prompt the user for the strings. Observe how the double square-bracket notation is used and the size of each buffer is in the brackets:

```
void getNames(char listNames[][256], int numNames)
{
   cout << "Who are your top " << numNames << " scripture heroes?\n";

   // standard FOR loop
   for (int iNames = 0; iNames < numNames; iNames++)
   {
      cout << "\t#" << iNames + 1 << ": ";
      cin  >> listNames[iNames];            // one element of an array of strings
   }                                        //    is simply a string!
}
```

To call the function, we specify that the entire list of names is to be used by passing the `names` variable without square brackets.

```
int main()
{
   // declare the array of strings
   char names[5][256];

   // prompt the user for the data
   getNames(names, 5 /*numNames*/);    // send the entire array of strings

   // display the list of names
   display(names, 5 /*numNames*/);

   return 0;
}
```

Challenge

As a challenge, modify the program so that each name only can contain 32 characters rather than 256. Also, change the list size to 10 items instead of five. How much code needs to change to make this work?

See Also

The complete solution is available at 3-0-arrayStrings.cpp or:

```
/home/cs124/examples/3-0-arrayStrings.cpp
```

Passing Characters, Strings, and List of Strings

Consider the following functions:

```
void displayListNames(const char names[][256], int num)  // second [] has the size
{
   for (int i = 0; i < num; i++)              // same as with other arrays
      cout << '\t' << names[i] << endl;       // access each individual string
}

void displayName(const char name[])  // no NUM variable, no value in the[]s
{
   cout << "One single name: " << name << endl;
}

void displayLetter(char letter)
{
   cout << "One single letter: " << letter << endl;
}
```

Given a list of names (`fullName`), we can call each of the above functions:

```
{
   char fullName[3][256] =
   {
      "Thomas",
      "Edwin",
      "Ricks"
   };

  displayListNames(fullName, 3);       // display all the members of fullName
  displayName(fullName[2]);            // display just the 3rd string in the list
  displayLetter(fullName[2][0]);       // display first letter of 3rd name
}
```

Given a single string (`word`), we can call only `displayName()` and `displayLetter()`:

```
{
   char word[256] = "BYU-Idaho";

   displayName(word);                     // pass a variable with the data “BYU-Idaho”
   displayLetter(word[4]);                // pass the letter ‘I’

   displayName("Vikings");                // pass a string literal “Vikings”
}
```

Finally, given a single letter (`letter`), we can call only `displayLetter()`:

```
{
   char letter = 'C';

   displayLetter(letter);                 // pass the variable ‘C’

   displayLetter('K');                    // pass the literal ‘K’
}
```

The complete solution is available at 3-0-passing.cpp or:

```
/home/cs124/examples/3-0-passing.cpp
```

Problem 1

What is the output?

```
{
   float nums[] = {1.9, 5.2, 7.6};

   cout << nums[1] << endl;
}
```

Answer:

_____

Please see page 218 for a hint.

Problem 2

What is the output?

```
{
   int a[] = {2, 4, 6};
   int b = 0;

   for (int c = 0; c < 3; c++)
     b += a[c];

   cout << b << endl;
}
```

Answer:

_____

Please see page 218 for a hint.

Problem 3

What is the output?

```
{
   char letters[] = "FFFFFFDCBAA";
   int  number = 87;

   cout << letters[number / 10]
        << endl;
}
```

Answer:

_____

Please see page 219 for a hint.

Problem 4

What is the output?

```
{
   int one[] = {6, 2, 1, 5};
   int two[] = {3, 9, 9, 7, 12};

   for (int i = 0; i < 2; i++)
      one[0] = one[i] * two[i];

   cout << one[0] << endl;
}
```

Answer:

_____

Please see page 218 for a hint.

Problem 5

What is the size of the following variables?

Declaration	Size of
`char a;` `cout << sizeof(a) << endl;`	
`char b[10];` `cout << sizeof(b) << endl;`	
`cout << sizeof(b[0]) << endl;`	
`int c;` `cout << sizeof(c) << endl;`	
`int d[20];` `cout << sizeof(d) << endl;`	
`cout << sizeof(d[1]) << endl;`	

Please see page 36 for a hint.

Problem 6

Write the code to put the numbers 1-10 in an array and display the array backwards:

Answer:

Please see page 220 for a hint.

Problem 7

Given the following function declaration:

```
void fill(int array[])
{
   for (int i = 0; i < 10; i++)
      array[i] = 0;
}
```

Write the code to call the function with a variable called `array`.

Answer:

Please see page 221 for a hint.

Problem 8

Given the following code:

```
{
   float numbers[32];

   display(numbers, 32);
}
```

Write a function prototype for `display()`.

Answer:

Please see page 223 for a hint.

Problem 9

Write the code to compute and display the average of the following numbers: 54.1, 18.6, 32.7, and 7:

```
Average is 28.1
```

Answer:

Please see page 220 for a hint.

Problem 10, 11

Write a function to prompt the user for 10 names. The resulting array should be sent back to `main()`.

Write a driver function `main()` to call the function and display the names on the screen.

Please see page 226 for a hint.

Assignment 3.0

This program will consist of three parts: `getGrades()`, `averageGrades()`, and a driver program.

getGrades

Write a function to prompt the user for ten grades and return the result back to `main()`. Note that any variables declared in `getGrades()` will be destroyed when the function returns. This means that `main()` will need to declare the array and pass it as a parameter to `getGrades()`.

averageGrades

Write another function to find the average of the grades and return the answer. Of course, the grades array will need to be passed as a parameter. The return value should be the average.

Driver Program

Finally, create `main()` that does the following:

- Has the grades array as a local variable
- Calls `getGrades()`
- Calls `averageGrades()`
- Displays the result

Please note that for this assignment, integers should be used throughout.

Example

The user input is **underlined**.

```
Grade 1: 90
Grade 2: 86
Grade 3: 95
Grade 4: 76
Grade 5: 92
Grade 6: 83
Grade 7: 100
Grade 8: 87
Grade 9: 91
Grade 10: 0
Average Grade: 80%
```

Assignment

The test bed is available at:

```
testBed cs124/assign30 assignment30.cpp
```

Don't forget to submit your assignment with the name "Assignment 30" in the header.

Please see page 49 for a hint.